

Dynamic Data Structures: Time and I/O Complexity

CMPT 211: Based on CLRS Algorithms (Chapter 10)

Bernal Manzanilla Saavedra

Concordia University of Edmonton

Winter 2026

The Scenario: The Dynamic Student Roster

We have successfully engineered our `ConcordiaStudent` objects. Now, the university needs a software system to hold all currently enrolled students.

This is a Dynamic Set. It changes constantly:

- **Search:** An administrator looks up Alice's GPA.
- **Insert:** A new student enrolls in the middle of the semester.
- **Delete:** Bob graduates and must be removed from the active roster.

*Question: How should we structure the memory to store these objects? A flat array? A Linked List? A Stack? The answer depends entirely on **Computational Complexity**.*

Evaluating Performance: Time vs. I/O Complexity

According to CLRS Chapter 10, we must evaluate data structures based on how many operations they require to maintain their integrity.

1. Time Complexity (Big O)

- How many iterations (loops) does the CPU have to execute to complete an operation?
- $O(1)$ is instant. $O(n)$ scales linearly with the number of students.

2. Memory I/O Complexity (Read/Write Operations)

- How many times do we have to physically read from or write to the computer's RAM?
- Writing to memory is "expensive" (slow). We want to minimize data movement!

The Cost of Standard Arrays (Python Lists)

Suppose we store 10,000 ConcordiaStudent objects in a standard, contiguous array, sorted alphabetically.

The Delete Operation: Alice drops out. We delete her object at Index 0.

```
roster = [alice_obj, bob_obj, charlie_obj, dave_obj, ... ]  
# We remove Alice from Index 0  
roster.pop(0)
```

The Hidden I/O Nightmare: Memory cannot have gaps! If Alice is removed from Index 0, the CPU must read Bob from Index 1 and write him to Index 0. Then read Charlie from Index 2 and write him to Index 1.

Deleting the first element of an array costs $O(n)$ Time and $O(n)$ Memory Writes! It is incredibly inefficient.

Stacks and Queues: $O(1)$ Complexity (CLRS 10.1)

Why did we learn about Stacks and Queues last week? To solve this exact I/O problem!

By strictly bounding where inserts and deletes can happen, we eliminate memory shifting entirely.

The Stack (LIFO):

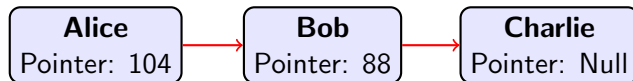
- Push: We write data exactly at the top pointer. (1 Write operation $\rightarrow O(1)$).
- Pop: We read data at the top pointer, and move the pointer down. (1 Read operation $\rightarrow O(1)$).

The Trade-Off: We achieved perfect $O(1)$ Insertion/Deletion, but we completely sacrificed **Search**. Finding a specific student in a Stack takes $O(n)$ time because we are only allowed to look at the top item!

Linked Lists: Decentralized Memory (CLRS 10.2)

What if we want $O(1)$ Insertions and Deletions *anywhere* in the data, not just at the ends?
We use a **Linked List**.

Instead of a contiguous array where neighbors must sit side-by-side, we scatter the ConcordiaStudent objects randomly throughout RAM. We just give each object a "Pointer" to tell the CPU where the next student is located.



Linked List Complexity: $O(1)$ Insert/Delete

The Magic of Pointers: If we want to insert a new student ("Dave") between Alice and Bob, we do NOT shift Bob or Charlie in memory.

We simply perform two $O(1)$ pointer writes:

- 1 We set Dave's pointer to look at Bob.
- 2 We overwrite Alice's pointer to look at Dave.

Zero memory shifting. We just inserted data into the middle of a million records in $O(1)$ Time and exactly 2 I/O Write operations!

The I/O Cost of Linked Lists

If Linked Lists have $O(1)$ Insertions and Deletions, why do we ever use Arrays?

The Search Penalty: Because the students are scattered randomly in memory, we cannot use math to jump to the middle (meaning Binary Search is impossible).

To find Charlie, the CPU must read Alice (1 I/O Read), extract the pointer, jump across the RAM hardware to read Bob (1 I/O Read), extract the pointer, and jump again to Charlie.

While Array searches are fast because contiguous memory is loaded into the CPU Cache in one giant block, Linked List searches trigger a high latency "Cache Miss" and a slow RAM I/O Read on every single hop! $O(n)$ Time, $O(n)$ I/O Reads.

Summary: CLRS Dynamic Set Operations

Data Structure	Search	Insert	Delete
Unsorted Array	$O(n)$	$O(1)$ (at end)	$O(n)$ (shifting)
Sorted Array	$O(\log n)$	$O(n)$ (shifting)	$O(n)$ (shifting)
Stack (LIFO)	$O(n)$	$O(1)$ (top)	$O(1)$ (top)
Queue (FIFO)	$O(n)$	$O(1)$ (tail)	$O(1)$ (head)
Linked List	$O(n)$	$O(1)$ (if pointer known)	$O(1)$ (if pointer known)

The Golden Rule of Algorithms: There is no perfect data structure. Every structure trades Time Complexity in one operation for I/O Efficiency in another!